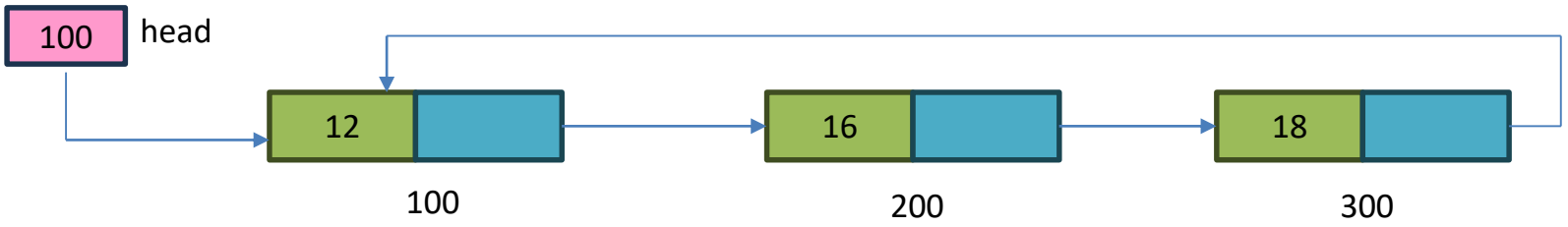
Circular Linked List

The circular linked list is a linked list where all nodes are connected to form a circle. In a circular linked list, the first node and the last node are connected to each other which forms a circle. There is no NULL at the end.



Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list.

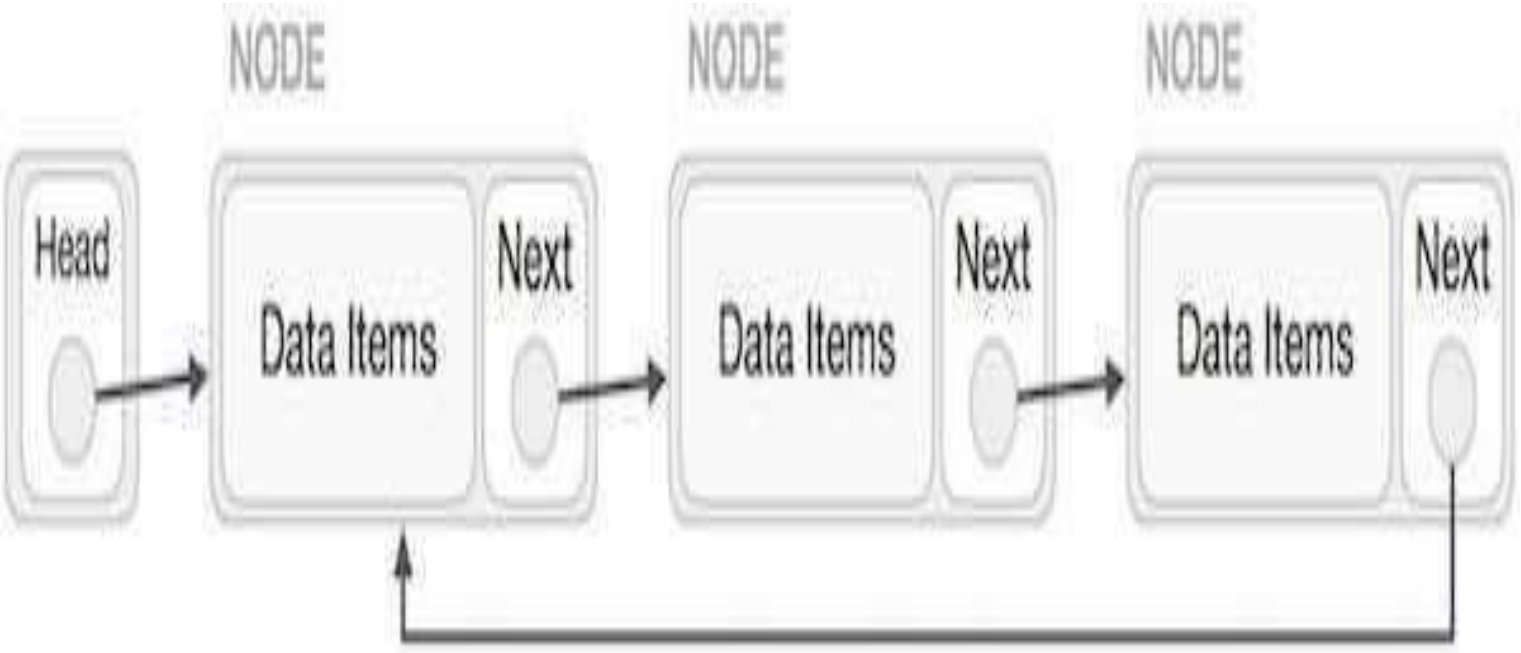
There are generally two types of circular linked lists:

•Circular singly linked list: In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We traverse the circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning or end. No null value is present in the next part of any of the nodes.

•Circular Doubly linked list: Circular Doubly Linked List has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by the previous and next pointer and the last node points to the first node by the next pointer and also the first node points to the last node by the previous pointer.

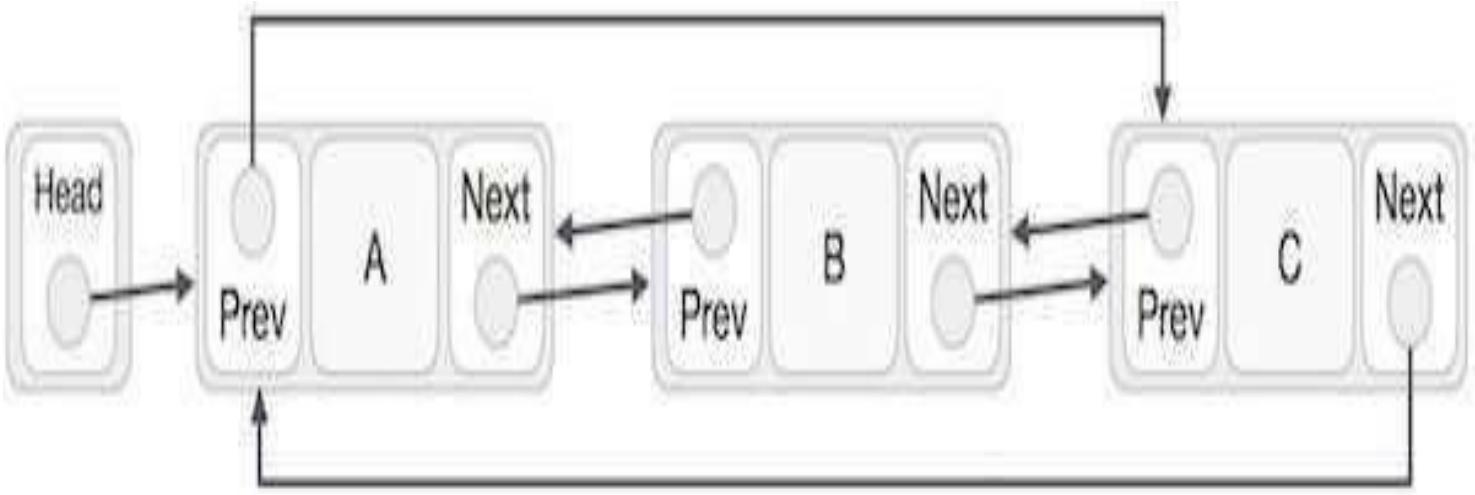
Singly Linked List as Circular

In singly linked list, the next pointer of the last node points to the first node.



Doubly Linked List as Circular

- In doubly linked list, the next pointer of the last node points to the first node and the previous pointer of the first node points to the last node making the circular in both directions.



Doubly Linked List as Circular

- As per the above illustration, following are the important points to be considered.
- The last link's next points to the first link of the list in both cases of singly as well as doubly linked list.
- The first link's previous points to the last of the list in case of doubly linked list.

Basic Operations

- Following are the important operations supported by a circular list.
- insert – Inserts an element at the start of the
- list.
- delete – Deletes an element from the start of
- the list.
- display – Displays the list.

Insertion In Circular Linked List

- There are three situation for inserting element in Circular linked list.
- Insertion at the front of Circular linked list.
- Insertion in the middle of the Circular linked list.
- Insertion at the end of the Circular linked
- list.

Insertion at the front of Circular linked list

Step1. Create the new node
Step2. Set the new node's next to itself (circular!)
Step3. If the list is empty,return new node.
Step4. Set our new node's next to the front.
Step5. Set tail's next to our new node.
Step6. Return the end of the list.

Algorithm for Insertion at the front of Circular linked list

```
node* AddFront(node* tail, int num)
 {
  node *temp = (node*)malloc(sizeof(node));
  temp->data = num;
  temp->next = temp;
  if (tail == NULL)
  return temp;
  temp->next = tail->next;
  tail->next = temp;
  return tail;
  }
```

Insertion at the end of Circular linked list

Step1. Create the new node
Step2. Set the new node's next to itself (circular!)
Step3. If the list is empty,return new node.
Step4. Set our new node's next to the front.
Step5. Set tail's next to our new node.
Step6. Return the end of the list.

Algorithm

- node* AddEnd(node* tail, int num)
- {
- node *temp = (node*)malloc(sizeof(node));
- temp->data = num;
- temp->next = temp;
- if (tail == NULL)
- return temp;
- temp->next = tail->next;
- tail->next = temp;
- return temp;
  }

Deletion in Circular singly linked list at the end

- There are three scenarios of deleting a node in circular singly linked list at the end.
- Scenario 1 (the list is empty)
- If the list is empty then the condition **head == NULL** will become true, in this case, we just need to print **underflow** on the screen and make exit.

Scenario 1

```
if(head == NULL)
    {
        printf("\nUNDERFLOW");
        return;
    }
```

- Scenario 2(the list contains single element)
- If the list contains single node then, the condition **head → next == head** will become true.
- In this case, we need to delete the entire list and make the head pointer free. This will be done by using the following statements.

Scenario 2

```
if(head->next == head)
{
    head = NULL;
     free(head);
}
```

- Scenario 3(the list contains more than one element)
- If the list contains more than one element, then in order to delete the last element, we need to reach the last node.
- We also need to keep track of the second last node of the list.

- For this purpose, the two pointers ptr and preptr are defined. The following sequence of code is used for this purpose.

```
ptr = head;
  while(ptr ->next != head)
  {
     preptr=ptr;
     ptr = ptr->next;
  }
```

Circular Linked Lists

Advantages of Circular Linked Lists

Efficient memory usage

The circular connectivity allows linked lists to reuse the same nodes, storing data in a fixed space. No dynamic resizing is needed. This makes circular lists useful for fixed-size buffers and queues.

Infinite traversal

The circular nature allows indefinite traversal of the list forward and backwards by following pointers. Useful for data requiring repeated access like game turns or audio playlists.

Constant time inserts/deletes

Adding or removing nodes simply requires reassigning a few pointers rather than shifting elements down or finding insertion points.

Preallocated memory

Circular lists allow the preallocation of nodes upfront since the size is fixed. This avoids the runtime overhead of dynamically requesting more memory.

Faster access than arrays

Retrieving the next or previous element is fast with pointer chasing. Arrays require calculating index offsets.

Hidden ends

The looped structure hides the actual start and end positions. Algorithms can treat any node as the first element.

History tracking

Circular lists provide an efficient way to store the history of events or states in a fixed buffer that gets overwritten after reaching capacity.

Looped/repeated processing

Good for modelling circular processes like round-robin scheduling. Nodes can be continuously traversed with no distinct end.

Bounded resource usage

Capping the list size bounds memory usage. Old data gets automatically overwritten by new after traversing full circle.

Simplified code

Algorithms on circular lists are simpler. No need for special handling for off-by-one errors or ends.

The main advantages are efficient memory usage, fast inserts and deletes, preallocation, infinite traversal, hidden ends, history tracking, and simplified circular algorithms. Circular lists shine when data exhibits a circular or looping pattern.

- **Applications of Circular Linked Lists**

  Ring Buffers
  Browser History
  Round Robin Scheduling
  Media Playlists
  Turn-based Games
  OS Process Scheduling
  Card Games
  Matrix Traversal
  Circular Buffers
  DNA Analysis